Darshan UNIVERSITY

योग: कर्मसु कौशलम्

Python Programming - 2301CS404

Lab - 15 (2)

Roll No.: 315

Name: Vishva M. Bhimani

Continued..

10) Calculate area of a ractangle using object as an argument to a method.

In [8]:
```
class Rectangle:
    def __init__(self,l,b):
        self.length=l
        self.breadth=b
    def area(self):
        return self.length*self.breadth
r=Rectangle(10,20)
print("Area =",r.area())
```

```
Area = 200
```

11) Calculate the area of a square.

Include a Constructor, a method to calculate area named area() and a method named output() that prints the output and is invoked by area().

In [12]:
```
class square:
    def __init__(self,side):
        self.s=side
    def area(self):
        self.result=self.s*self.s
```

```
        self.output()
    def output(self):
        print("area=",self.result)
s=square(2)
s.area()
```

```
area= 4
```

12) Calculate the area of a rectangle.

Include a Constructor, a method to calculate area named area() and a method named output() that prints the output and is invoked by area().

Also define a class method that compares the two sides of reactangle. An object is instantiated only if the two sides are different; otherwise a message should be displayed : THIS IS SQUARE.

In [20]:
```
class Rectangle:
    def __init__(self,l,b):
        self.length=l
        self.breadth=b

    @classmethod
    def compare(cls,l,b):
        if l==b:
            print("This is Square")
            return None
        else:
            return cls(l,b)

    def area(self):
        return self.length*self.breadth
        self.output()

    def output(self):
        print("Area: ",self.result)

r= Rectangle.compare(10,10)
print(r)
r1=Rectangle.compare(5,10)
r1.area()
```

```
This is Square
None
```

Out[20]: 50

13) Define a class Square having a private attribute "side".

Implement get_side and set_side methods to accees the private attribute from outside of the class.

In [21]:
```
class Square:
    def __init__(self,side):
        self.__side=side
    def get_side(self):
        return self.__side
    def set_side(self,value):
        self.__side = value
s = Square(5)
print("Side:", s.get_side())
s.set_side(10)
print("Updated Side:", s.get_side())
```

```
Side: 5
Updated Side: 10
```

14) Create a class Profit that has a method named getProfit that accepts profit from the user.

Create a class Loss that has a method named getLoss that accepts loss from the user.

Create a class BalanceSheet that inherits from both classes Profit and Loss and calculates the balanace. It has two methods getBalance() and printBalance().

In [24]:
```
class Profit:
    def getProfit(self):
        self.profit=float(input("Enter Profit: "))
class Loss:
    def getLoss(self):
        self.loss=float(input("Enter Loss: "))
class BalanceSheet(Profit,Loss):
    def getBalance(self):
        self.balance=self.profit-self.loss
    def printBalance(self):
        print("Balance: ",self.balance)
b = BalanceSheet()
b.getProfit()
b.getLoss()
b.getBalance()
b.printBalance()
```

```
Balance:  90.0
```

15) WAP to demonstrate all types of inheritance.

In [25]:
```
class A:
    def showA(self):
        print("Class A")
class B(A):
    def showB(self):
        print("Class B")
```

```
class C(B):
    def showC(self):
        print("Class C")

class X:
    def showX(self):
        print("Class X")
class Y:
    def showY(self):
        print("Class Y")
class Z(X, Y):
    def showZ(self):
        print("Class Z")
obj = C()
obj.showA()
obj.showB()
obj.showC()
obj2 = Z()
obj2.showX()
obj2.showY()
```

```
Class A
Class B
Class C
Class X
Class Y
```

16) Create a Person class with a constructor that takes two arguments name and age.

Create a child class Employee that inherits from Person and adds a new attribute salary.

Override the **init** method in Employee to call the parent class's **init** method using the super() and then initialize the salary attribute.

In [30]:
```
class Person:
    def __init__(self,name,age):
        self.n=name
        self.a=age

class Employee(Person):
    def __init__(self,name,age,salary):
        self.s=salary
        super().__init__(name,age)
        self.s=salary

e1=Employee("Jiya",20,1000000)
print(e1.n)
print(e1.a)
print(e1.s)
```

```
Jiya
20
1000000
```

17) Create a Shape class with a draw method that is not implemented.

Create three child classes Rectangle, Circle, and Triangle that implement the draw method with their respective drawing behaviors.

Create a list of Shape objects that includes one instance of each child class, and then iterate through the list and call the draw method on each object.

In [31]:
```python
class Shape:
    def draw(self):
        pass

class Rectangle(Shape):
    def draw(self):
        print("Drawing reactangle")

class Circle(Shape):
    def draw(self):
        print("Drawing circle")

class Triangle(Shape):
    def draw(self):
        print("Drawing Triangle")

shape=[Rectangle(),Circle(),Triangle()]

for shape in shape:
    shape.draw()
```

```
Drawing reactangle
Drawing circle
Drawing Triangle
```

In [ ]:
```python
```